

/pydantic evals

LLM-as-a-Judge: A Practical Guide with Pydantic Evals



David Montague

📖 12 mins 2026/02/11

TL;DR

- **Evaluation is easier than generation** — the judge sees both question and answer, making it a much narrower task than open-ended generation
- **Case-specific evaluators outperform generic ones for test suites** — if an LLM could reliably assess quality without case context, it could probably generate good responses in the first place
- **Combine deterministic checks with LLM judges strategically** — run type validation and format checks first; save LLM evaluation for semantic quality
- **Always request reasoning** — it's essential for debugging failures and iterating on rubrics
- **Connect user feedback to test cases** — every complaint is a potential evaluation case that proves you've fixed the issue

Introduction

Evaluating LLM outputs at scale is one of the hardest problems in AI engineering. Human evaluation doesn't scale. Traditional metrics like BLEU and ROUGE miss semantic quality

entirely. You need something that understands language, applies nuanced criteria, and runs automatically.

Enter LLM-as-a-Judge: using one language model to evaluate another's output. It sounds circular, but it works, and for reasons worth understanding. This guide covers what you need to implement it effectively, including an important distinction most content on this topic misses: the difference between one-size-fits-all and case-specific evaluators. We'll show when to use each, how to write effective rubrics, and how to integrate evaluation into your workflow.

All examples use [pydantic-evals](#) for implementation and [Logfire](#) for observability.

What is LLM-as-a-Judge?

LLM-as-a-Judge uses a language model to assess the quality of another LLM's output. Instead of exact string matches or statistical metrics, you give the judge a rubric describing what "good" looks like, and it returns a verdict.

The pattern:

1. Your task receives an input and produces an output
2. The judge receives the output (and optionally the input and expected answer)
3. The judge applies your rubric to assess quality
4. The judge returns a verdict: pass/fail, a score, or a label

Here's the simplest possible example with pydantic-evals:

```

"""Basic LLM-as-a-Judge example showing how to evaluate a support
bot."""
import asyncio

from pydantic_evals import Case, Dataset
from pydantic_evals.evaluators import LLMJudge

dataset = Dataset(
    cases=[
        Case(
            name='password_reset_help',
            inputs='How do I reset my password?',
        ),
    ],
    evaluators=[
        LLMJudge(rubric='The response provides clear, actionable
instructions'),
    ],
)

async def support_bot(query: str) -> str:
    """A simple support bot that returns canned responses."""
    return (
        "To reset your password, click 'Forgot Password' on the login
page, "
        "enter your email, and follow the link we send you."
    )

async def main():
    report = await dataset.evaluate(support_bot)
    report.print()

if __name__ == '__main__':
    asyncio.run(main())

```

The LLMJudge evaluator sends your rubric and the task output to an LLM (GPT-4o by default), which returns whether the output passes and an explanation of its reasoning.

Why It Works

The core insight is that **evaluation is easier than generation**.

When generating a response, the LLM faces an enormous solution space. It must satisfy many constraints simultaneously: be accurate, helpful, match the right tone, stay on topic, avoid harmful content. The solution space is vast, and the model is navigating it alone.

When evaluating, the task is fundamentally narrower. The judge sees both the question and the answer. It doesn't have to produce anything; it just has to look at what's already there and decide if it's any good. Given a specific rubric like "Does this response answer the user's question?", the success criteria are clear. It's always easier to critique than to create.

Research bears this out. Studies on MT-Bench and Chatbot Arena show 80%+ agreement between LLM judges and human preferences on many tasks. Agreement is highest for clear-cut criteria and lowest for subjective judgments, which is exactly what you'd expect.

Perhaps most surprisingly, **the same model that generated a response can often catch its own mistakes** when asked to evaluate. This sounds paradoxical, but generation and evaluation really are different cognitive tasks. A model might hallucinate a fact during generation (it doesn't know what it doesn't know), but when you ask it to compare its claims against source material, that's a much simpler operation. We all have better judgment in hindsight.

Of course, you *can* build this kind of iterative self-analysis into an agent's generation loop, capturing much of the value of an LLM judge at inference time. But in practice, you're often trying to reduce token usage or improve latency, which means you want good responses on the first try. That's where offline evaluation earns its keep.

The Key Distinction: One-Size-Fits-All vs. Case-Specific

This is the most important idea in this post, and the one most writing about LLM-as-a-Judge skips.

One-Size-Fits-All Evaluators

A **one-size-fits-all evaluator** uses a single rubric applied uniformly to every test case:

```
from pydantic_evals import Case, Dataset
from pydantic_evals.evaluators import LLMJudge

dataset = Dataset(
    cases=[
        Case(name='billing_question', inputs='Why was I charged
twice?'),
        Case(name='feature_request', inputs='Can you add dark mode?'),
        Case(name='bug_report', inputs='The app crashes when I upload
photos'),
    ],
    # This evaluator runs on ALL cases with the same rubric
    evaluators=[
        LLMJudge(
            rubric='Response is professional, empathetic, and does not
blame the user',
            include_input=True,
        ),
    ],
)
```

The rubric must be general enough to apply everywhere. This works well for quality dimensions that genuinely are universal: professionalism, safety, tone. These are especially useful for evaluating *orthogonal* concerns, things that should always be true regardless of what the LLM was actually asked to do.

Case-Specific Evaluators

A **case-specific evaluator** has a rubric tailored to an individual test case:

```

from pydantic_evals import Case, Dataset
from pydantic_evals.evaluators import LLMJudge

dataset = Dataset(
    cases=[
        Case(
            name='vegetarian_recipe',
            inputs='I need a vegetarian dinner recipe',
            evaluators=[
                LLMJudge(
                    rubric=''
                    Recipe must NOT contain meat, meat-based broths,
                    gelatin, or other animal-derived ingredients.
                    PASS only if the recipe is fully vegetarian.
                    '',
                    include_input=True,
                ),
            ],
        ),
        Case(
            name='quick_weeknight_meal',
            inputs='I need something I can make in under 30 minutes',
            evaluators=[
                LLMJudge(
                    rubric=''
                    Recipe must have total prep + cook time under 30
minutes,
                    use commonly available ingredients, and not require
                    specialized equipment.
                    FAIL if the recipe would realistically take longer.
                    '',
                    include_input=True,
                ),
            ],
        ),
        Case(
            name='allergy_safe',
            inputs='Nut-free dessert for a school event',
            evaluators=[
                LLMJudge(
                    rubric=''
                    Recipe must NOT contain tree nuts, peanuts, or

```

```

        peanut-derived ingredients. This is for a child
        with allergies. FAIL if any nut risk exists.
        '''
        include_input=True,
    ),
],
),
],
# Universal evaluator still runs on all cases
evaluators=[
    LLMJudge(
        rubric='Recipe instructions are clear and easy to follow',
        include_input=True,
    ),
],
)

```

Notice how each case has requirements that would be impossible to squeeze into a single universal rubric. The vegetarian case couldn't care less about cooking time. The quick meal case has no opinions about allergens. Trying to write one rubric that captures all of this would produce something vague and difficult to maintain. You'd end up spending real effort evaluating whether your evaluator is even working correctly. With case-specific rubrics, the rubric itself is simple enough that you can trust it on inspection.

Why This Matters

Here's the realization that ties it together:

If an LLM were good enough to assess quality reliably across all cases without context, it would likely be good enough to generate good responses in the first place.

This is especially true when evaluating agents that can reflect on their own output, or when using thinking models for generation. In those cases, a generic LLM rubric would only catch failures the model could have avoided by thinking one step longer. Case-specific evaluators sidestep this by providing context the judge needs that the generator never had:

- One-size-fits-all: "Is this a good essay?" (vague, hard to judge)
- Case-specific: "Does this essay argue that renewable energy is cost-effective, using at

least three economic studies?" (clear, verifiable)

That said, one-size-fits-all evaluators have a compelling use case beyond style and safety checks: **validating cheaper or faster models**. If you're using a smaller model in production to save cost or reduce latency, a more capable judge model can confirm the cheaper one is producing adequate results. For many tasks, a flagship model is overkill, but it's good to verify that. Running a capable judge over a sample of your cheaper model's outputs is a practical way to gain that confidence.

When to Use Each

Scenario	Approach	Rationale
Online production monitoring	One-size-fits-all	No case-specific context available at runtime
Building a test suite	Case-specific preferred	Capture nuanced expectations per scenario
Universal quality checks (tone, safety)	One-size-fits-all	These truly apply to all outputs
Validating cheaper/faster models	One-size-fits-all	Capable judge confirms adequate quality
Regression testing after changes	Case-specific	Verify specific behaviors are preserved

Where LLM Judges Excel

Groundedness and Hallucination Detection

LLM judges are particularly effective at detecting hallucinations because the task is well-defined: "Is claim X supported by source Y?"

```

from pydantic import BaseModel
from pydantic_evals import Case, Dataset
from pydantic_evals.evaluators import LLMJudge


class RAGInput(BaseModel):
    question: str
    retrieved_context: str


dataset = Dataset(
    cases=[
        Case(
            name='policy_question',
            inputs=RAGInput(
                question='What is the return policy?',
                retrieved_context=''
                Items can be returned within 30 days of purchase.
                Items must be unused and in original packaging.
                Refunds are processed within 5-7 business days.
                Sale items are final sale.
                ''',
            ),
            evaluators=[
                LLMJudge(
                    rubric=''
                    The response must ONLY contain information from the
                    retrieved context. FAIL if the response includes
ANY
                    information not in the context – even if it seems
plausible.
                    ''',
                    include_input=True,
                ),
            ],
        ),
    ],
)

```

The `include_input=True` is crucial here. It gives the judge the source material to check against. Without it, you're asking someone to spot lies without telling them what the truth is.

Style and Tone

Style checks cross-cut other quality dimensions and make excellent one-size-fits-all evaluators:

```
from pydantic_evals.evaluators import LLMJudge

style_evaluators = [
    LLMJudge(
        rubric='Response uses second person ("you", "your") instead of third person',
        assertion={'evaluation_name': 'uses_second_person'},
    ),
    LLMJudge(
        rubric='''Response avoids corporate jargon: "leverage", "synergy", "paradigm", "circle back", "move the needle", etc.''' ,
        assertion={'evaluation_name': 'no_jargon'},
    ),
]
```

These are clear, binary criteria that apply universally and are hard to check programmatically (regex would be brittle).

Hard-to-Generalize Quality Rules

Some criteria are easy to articulate for a specific case but hard to write as universal rules:

- "This coding question needs a solution that handles the edge cases the user mentioned"
- "This customer is frustrated, so the response needs to be especially empathetic"
- "This is a follow-up question, so the response shouldn't repeat context already established"

Case-specific LLM judges handle these naturally.

Where NOT to Use LLM Judges

When Deterministic Checks Will Do

This should go without saying, but: **if you can check it with code, check it with code.**

- **Type validation:** Use Pydantic models
- **Format validation:** JSON schema, regex patterns
- **Length constraints:** Character or word counts
- **Exact value matching:** String equality, numeric comparisons

Deterministic checks are faster (milliseconds vs. seconds), cheaper (free vs. API calls), perfectly consistent, and much easier to debug.

```
from pydantic_evals import Case, Dataset
from pydantic_evals.evaluators import LLMJudge, IsInstance

dataset = Dataset(
    cases=[
        Case(name='refund_request', inputs='I want a refund for my
order'),
    ],
    evaluators=[
        # Fast, deterministic check first
        IsInstance(type_name='str'),
        # Semantic check that needs LLM judgment
        LLMJudge(
            rubric='Response acknowledges the refund request and
provides next steps',
            include_input=True,
        ),
    ],
)
```

Structure your evaluators from cheapest to most expensive. If the response isn't even the right type, there's no point asking a language model whether it's *helpful*.

The Nuance: Brittle vs. Robust Matching

Checking for specific words *feels* deterministic but is often brittle. A naive 'refund' in response check fails because of synonyms ("reimbursement", "money back"), negations ("We cannot offer a refund" contains "refund" but means the opposite), and context differences. For semantic matching, LLM judges are more robust.

But sometimes exact matching is exactly right: verifying a calculated value appears verbatim, checking for required legal disclaimers, or ensuring a specific product SKU is mentioned.

Writing Effective Rubrics

The rubric is everything. A lazy rubric produces lazy evaluations, and no amount of model capability will save you from asking the wrong question.

Level	Example	Problem
Bad	"Response is good"	What does "good" mean?
Okay	"Response is helpful"	Helpful how?
Better	"Response answers the user's question"	What counts as answering?
Good	"Response directly addresses the question with actionable next steps"	Clear criteria

Best	Includes what to look for AND what constitutes failure	Unambiguous
------	--	-------------



Pydantic

now it when I see it" to "here are the
fy both what success looks like and

what failure looks like, leaving the judge very little room to improvise:

```
from pydantic_evals.evaluators import LLMJudge

good_judge = LLMJudge(
    rubric=''
    The response must:
    1. Directly address the user's question (not a tangential topic)
    2. Provide at least one actionable next step
    3. Not require the user to ask follow-up questions for basic
    information

    PASS if all three criteria are met.
    FAIL if any criterion is not met, and explain which one(s).
    '',
    include_input=True,
    assertion={'evaluation_name': 'helpful', 'include_reason': True},
)
```

Binary vs. Numeric

Use binary (pass/fail) when the world really is black and white: policy compliance, safety violations, the presence or absence of PII. These are not matters of degree.

Use numeric scores when quality exists on a spectrum, you want to track improvement over time, or you need to rank responses.

```

from pydantic_evals.evaluators import LLMJudge

# Binary assertion
assertion_judge = LLMJudge(
    rubric='Response follows company guidelines',
    assertion={'evaluation_name': 'compliant', 'include_reason': True},
    score=False,
)

# Numeric score
score_judge = LLMJudge(
    rubric='Rate the helpfulness of the response from 0 to 1',
    score={'evaluation_name': 'helpfulness', 'include_reason': True},
    assertion=False,
)

```

Context Levels

Three levels of context you can give the judge:

- **Output only** (default): For style checks and format validation
- **Input + Output** (`include_input=True`): For relevance, completeness, appropriateness
- **Input + Output + Expected** (`include_input=True` , `include_expected_output=True`): For correctness and semantic equivalence

Give the judge the minimum context it needs. More context can actually hurt by distracting from the criteria that matter.

Always Request Reasoning

```
from pydantic_evals.evaluators import LLMJudge

judge = LLMJudge(
    rubric='Response is accurate',
    assertion={'evaluation_name': 'accurate', 'include_reason': True},
)
```

When a case fails, the reason tells you *why*. Without it, you're staring at a red light with no idea what tripped it. The reasoning lets you debug, verify the judge actually applied your rubric correctly (they don't always), and iterate on your rubrics based on how the judge interprets them.

From User Feedback to Test Cases

Every user complaint is a gift, though it rarely feels like one at the time. When someone reports an issue, they're handing you a test case: a specific scenario where your system failed, with a clear description of what went wrong.

```

from pydantic_evals import Case
from pydantic_evals.evaluators import LLMJudge

# User ticket: "Bot denied my refund, but I bought it 2 weeks ago and
it's broken"
Case(
    name='refund_eligibility_false_negative',
    inputs='I bought this 2 weeks ago and it broke. Can I get a
refund?',
    metadata={'source': 'user_feedback', 'ticket_id': '12345'},
    evaluators=[
        LLMJudge(
            rubric=''
            Context: Purchase was 2 weeks ago (within 30-day window).
            Product is defective.

            Response MUST confirm the customer is eligible for a refund
            and provide return instructions.
            FAIL if refund is denied or eligibility is unclear.
            '',
            include_input=True,
        ),
    ],
)

```

Each complaint becomes a test case. The rubric captures what went wrong. Passing this case proves the issue is fixed. Over time, your evaluation suite captures the full range of user expectations.

The cycle: **Collect** feedback. **Convert** it to test cases. **Evaluate** against current and candidate prompts. **Deploy** improvements that pass the new cases without regressing on the old ones. **Repeat**.

One thing: don't delete test cases after improving the prompt. The case passes now, so the temptation is to clean it up. But prompts change. Models change. The case that passes today might fail next month after an unrelated edit. You're building a cumulative test suite, and that means keeping the history.

Choosing Your Evaluator

Situation	Evaluator Type	Example
Universal quality check	One-size-fits-all LLMJudge	"Response is professional"
Specific requirements per scenario	Case-specific LLMJudge	"Recipe has no gluten"
Type/format validation	Deterministic (<code>IsInstance</code>)	Output is a Pydantic model
Semantic presence check	LLMJudge	"Mentions the refund policy"
Exact presence check	Deterministic (<code>Contains</code>)	Specific value appears
Comparing to source material	LLMJudge with <code>include_input</code>	Hallucination detection
Comparing to expected answer	LLMJudge with <code>include_expected_output</code>	Correctness check

Resources

- [pydantic-evals documentation](#)
- [pydantic-ai documentation](#)

- [Logfire](#)
- [Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" \(2023\)](#)

Related content

[View all articles](#)

/Pydantic AI

The runtime layer underneath your Pydantic AI agent

Hamza Tahir

2026/05/25

/Pydantic AI

Build durable agents with Restate and Pydantic AI

Giselle van Dongen

2026/05/12



Explore Logfire

Get started

Explore our [open source](#) packages

`'/logfire'`

`'/gateway'`

`'/about'`

`'/careers'`

`'/privacy'`

`'/cookies'`

`'/blog'`

`'/pricing'`

`'/security'`

`'/pydantic-ai'`

`'/pydantic'`

`'/contact'`

`'/newsletter'`

`'/terms'`

`'/casestudies'`

`'/opensource'`

`'/enterprise'`

`'/alternatives'`

'/vs-langsmith'

'/vs-langfuse'

'/vs-arize'

The Pydantic Stack

Sign up for updates on Pydantic AI, Pydantic Logfire, and Pydantic Evals.

Enter your email address

Subscribe



Pydantic